

Assignment 1

Image Classification with Neural Networks

Teva Philippe & Alvaro Crespo

1. Introduction

The objective of this project is to analyze and compare the performance of a neural network on an image classification dataset. We first built a fully connected network (MLP), then improved it by adding convolution layers, and looked for the best parameters for our model (hidden layers, convolution, activation function, optimizer, scheduler and regularization). Finally, we used this model to analyze how it behaves on different and similar sports, on color and grayscale images, and on a growing number of classes.

2. Dataset Analysis

The dataset we are using for this project is the 100 Sports Image Classification dataset. A dataset of thousands of images from different sports, in order to solve classification problems.

We decided to choose this specific dataset, because with 100 classes, we could pick and choose different amounts of classes based on our needs, but also test how the amount of classes has an impact on the model's ability to learn



Fig. 1. – Example of some of the images found in the dataset.

2.1. Data Quality & Characteristics

The dataset includes 14492 images, divided into 100 different classes. The main directory has three folders and two files:

- train/ has 13493 files
- test/ has 500 files
- valid/ has 500 files
- sports.csv
- EfficientNetB0-100-(224 X 224)- 98.40.h5

The files in the three folders are all JPG images, and are equally sized (224x224 pixels). The class distribution is not balanced, with the biggest class (football) being 2.91 times bigger than the smallest class (sky surfing).

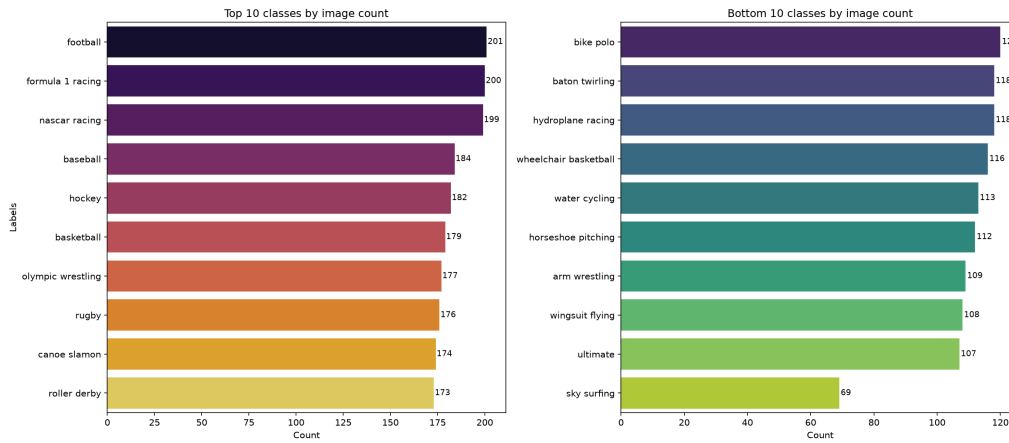


Fig. 2. – Class distribution (10 highest and 10 lowest) for the dataset.

The CSV file (sports.csv) contains basic information for all the images:

class id	filepaths	labels	data set
0	train/air hockey/001.jpg	air hockey	train
0	train/air hockey/002.jpg	air hockey	train
0	train/air hockey/003.jpg	air hockey	train
0	train/air hockey/004.jpg	air hockey	train
0	train/air hockey/005.jpg	air hockey	train

.head() of sports.csv.

And the HDF5 file (EfficientNetB0-100-(224 X 224)- 98.40.h5) is not mentioned in the dataset documentation, but is assumed to be a pre-trained model that can be used for tasks such as transfer learning or quick predictions, but that goes beyond the scope of this project, so we are not going to use this file.

The author of the dataset claims that « This dataset was carefully created so that the region of interest (ROI) in this case the sport occupies approximately 50% of the pixels in the image. As a consequence even models of moderate complexity should achieve training and validation accuracies in the high 90's. »

For our experiments, we decided to reduce the amount of classes in order to carry out our different analyses. We started with three different classes, and then tested some different combinations:

- 3 very different classes, with the objective of creating an « starting point ».
- 3 similar classes, to test model's ability to learn finer details.
- 6, 12 and 20 classes, to see how the model handles more classes
- The whole dataset (100 classes), to “challenge” our model.

*Class choice for each experiment was based on class distribution, prioritizing classes with higher number of images.

3. Model description

3.1. Data loading and preparation

- The dataset is downloaded with kagglehub, which stores it in a cache folder on the machine.

- Each image goes through a transformation: we kept their original size of 224×224 pixels, then converted them to tensors, with pixel values between 0 and 1. For the grayscale version, a Grayscale step is added: the image then has only one channel instead of three (red, green, blue), which divides the number of input values by three.

A 64×64 pixel version was also prepared for a later test.

- A SubsetImageFolder class (generated) allows us to keep only some sports:

First, we selected 3 similar sports and 3 different sports.

3 different sports	3 similar sports
Football (191 files)	Football (191 files)
Nascar racing (189 files)	Rugby (166 files)
Olympic wrestling (167 files)	Ampute football (112 files)

Table A : Sport choosen and number of files they contains

We also chose a subset of 6 different sports, then subsets of 12 and 20 sports, some of them different and some of them similar.

3.2. The model

The Net class is an MLP (fully connected neural network) for which we can choose:

- the number and size of the hidden layers (hidden_sizes, for example [128, 64]);
- the number of output classes;
- the activation function, among the three seen in class: **ReLU**, **tanh** and **sigmoid**;
- the regularization (see below);
- an optional convolution part placed before the MLP.

If no convolution is given, the image is simply flattened into a long vector of pixels, then sent to the MLP. Otherwise, the image first goes through the convolution, and the result is then flattened.

3.3. The convolution

- The Conv class contains a convolution layer, a ReLU, then a 2×2 pooling. The padding keeps the same image size. We can choose the number of input channels (1 for grayscale, 3 for color), the number of filters, the kernel and the type of pooling (**max** or **average**).
- The Conv2 class applies two Conv blocks one after the other. The second block uses twice as many filters as the first one.

3.4. Regularizations

Four regularizations are available, chosen with the regularization parameter:

- **L1**, **Weight decay**, **Dropout** and **Early stopping**

3.5. Optimizers and scheduler

Two optimizers are available:

- **SGD** and **Adam**

A **Step** scheduler can be added: it divides the learning rate by 2 every 10 epochs.

3.6. Training and visualization

- **train**: at each epoch, the model is trained on the training set and then evaluated on the validation set. The function records the loss and the accuracy (training and validation) as well as the confusion matrix of each epoch.
- **run_training**: creates the DataLoaders (batches of 64 images), builds the convolution, creates the model and starts the training.
- **plot_metrics**: displays the accuracy and loss curves (training and validation) and the confusion matrices of the requested epochs.
- **nb_parameters**: counts the number of parameters of the model.

Note that the seed is set to 42 to ensure that the results are comparable.

4. Process and Analysis

For the process, we looked for the ideal parameters on a subset of **12 sports in grayscale**, adding the elements one by one.

Once these parameters were found, we applied them to our other subsets. The idea was to compare the behaviour of such a model on 3 different sports and on 3 similar sports, between the use of grayscale images and color images (we will see an example with 6 sports), and finally with a growing number of classes (20 and 100 sports).

To compare two models, we mainly look at the **validation loss**. The validation accuracy is computed on a few images (5 images per class): it therefore varies a lot from one epoch to another.

4.1. Choice of the hidden layers

We started without convolution, with SGD, a learning rate of 0.001 and 20 epochs: the image is simply flattened and then given to the MLP.

Hidden layers	Parameters	Minimum validation loss
128	6.4 M	2.11
512, 128	25.8 M	2.12
128, 64	6.4 M	2.13
256, 128	12.9 M	2.16
64, 32	3.2 M	2.18
128, 64, 32	6.4 M	2.24

Table B : Number of parameters and Loss in relation with the hiddens layers configuration

The results are very close. The models are still learning at the end of the 20 epochs. Our model was running on the GPU of the virtual machines, but given the number of images and parameters, it still took a long time to run. This is why we stayed at 20 epochs. We kept **two layers of 128 and 64 neurons**, a good compromise between results and number of parameters.

4.2. Adding the convolution

We then added a convolution layer (Conv), testing two filter sizes (3 and 7) and three numbers of filters (8, 16 and 32).

Kernel	8 filters	16 filters	32 filters
3	2.01	1.95	1.95
7	2.01	1.90	1.91

Table C : Loss in relation with kernel size and number of filters

The convolution improves the results compared to the MLP alone. Going from 8 to 16 filters helps a little, but not from 16 to 32. Kernels of 3 and 7 give very similar results. However, the number of parameters increases very quickly with the number of filters (12.9 M with 8 filters, 25.7 M with 16, 51.4 M with 32). We therefore kept **8 filters and a kernel of 3** to stay at 12.9 M parameters.

We also compared **max pooling** and **average pooling**, which were equivalent, and then the activation functions: ReLU (2.01) and tanh (1.99) give almost identical results, while sigmoid learns nothing (the loss stays at the level of a model answering at random). We kept max pooling and ReLU.

Given the results, we could have kept tanh, but we know from theory that ReLU is better, so we preferred not to take any risks.

4.3. Optimizer and scheduler

With SGD and 40 epochs, the model starts to overfit: the training loss keeps decreasing, while the validation loss stays around 2. At first, we started to regularize at this point. But we ran into difficulties that we will come back to later. This is why, in the end, we went directly to the optimizer and scheduler part.

We replaced SGD with Adam, which adapts the weights, with the step scheduler, which divides the learning rate by 2 every 10 epochs.

Configuration	Minimum validation loss	Validation accuracy (end)
SGD	1.86	about 33 %
Adam + scheduler step	1.64	about 50 %

Table D : Loss and accuracy in relation with Optimizer and Scheduler

The improvement is clear, and the validation loss is much more stable at the end of training, which matches the expected effect of the scheduler.

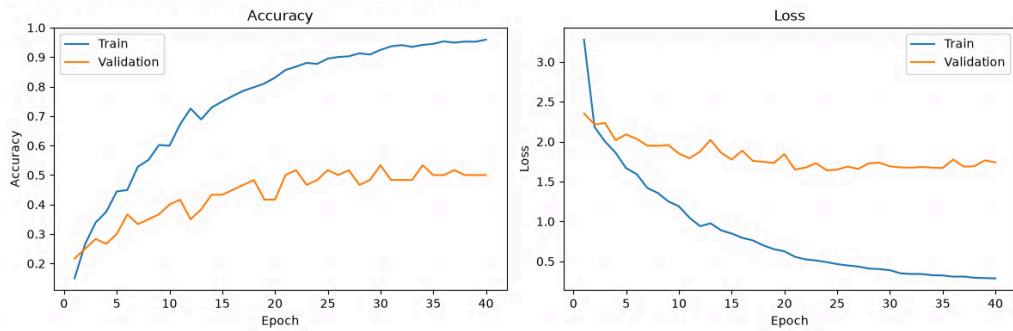


Fig. 3. – 12 sports in grayscale, Adam + scheduler: the model learns better, but the gap between training and validation remains large.

4.4. Regularization: no improvement

The model still overfits: it reaches almost 100 % on the training set, but about 50 % on the validation set. We therefore tested the regularizations, with Adam and the scheduler.

Regularization	Value	Minimum validation loss
None	–	1.64
Weight decay	0.0001	1.62
Weight decay	0.001	1.66
Weight decay	0.01	1.77
Weight decay	0.1	2.08
Dropout	0.1	1.78
Dropout	0.3	2.13
L1	0.00005	1.82
L1	0.0001	1.90
L1	0.0003	2.01

Table E : Value applied for each regularization and Loss obtained

No regularization really improves the model, or only very slightly. With stronger values, the gap between training and validation decreases, but the model learns less well, and with 0.1 or a dropout of 0.3, we turned overfitting into underfitting. Dropout leads to the same conclusion as weight decay.

We also tried reducing the image size to 64×64 pixels. In that case, the model had far fewer parameters (1 M instead of 12.9 M) and overfitted less, but the validation loss was worse (1.87). By reducing the images, we lost details that are useful to recognize the sports.

4.5. A second convolution

We then decided to add a second convolution layer (Conv2). Each pooling divides the image size again, so the first layer of the MLP becomes smaller (6.4 M parameters instead of 12.9 M), while keeping the images at 224×224.

This is the clearest improvement: the minimum validation loss goes from 1.64 to **1.33** (around epoch 11), and the validation accuracy rises to about 55 %. On the other hand, the model learns much faster and reaches 100 % on the training set from around epoch 12. After that, the validation loss goes back up.

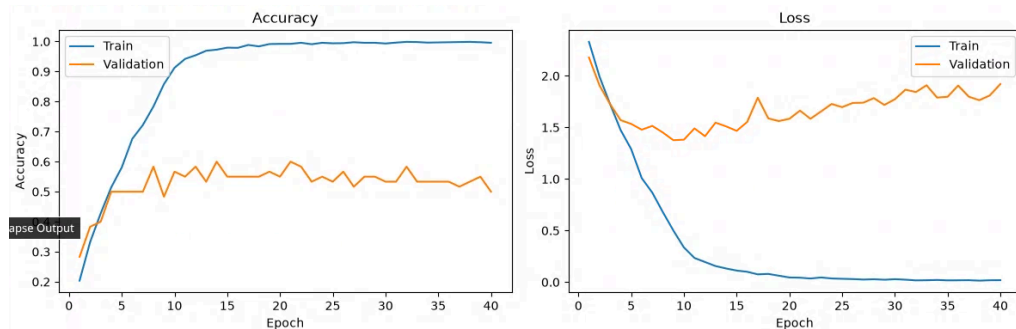


Fig. 4. – 12 sports in grayscale with two convolutions: better results, but fast overfitting after epoch 11.

We tried again to regularize this model with dropout (0.1), without improvement.

We therefore chose to use **early stopping** to stop training around the best moment. Our early stopping has a limitation: it stops 10 epochs after the best validation loss.

Our final configuration is therefore: two convolutions (8 then 16 filters, kernel of 3, max pooling), an MLP of 128 and 64 neurons with ReLU, Adam with the scheduler, and early stopping.

4.6. Comparison of the subsets

We applied this final configuration to all our subsets, in grayscale and in color.

4.6.1. 3 different sports or 3 similar sports

These two subsets contain the same number of classes. The only difference is how similar the sports are, and this is the impact we wanted to compare.

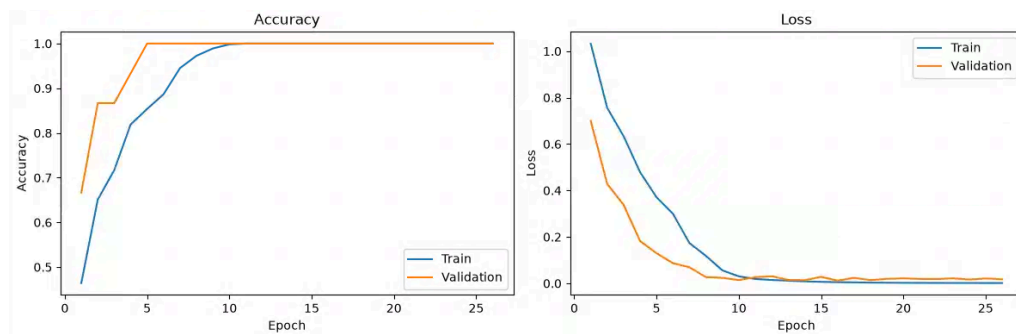


Fig. 5. – 3 different sports (football, nascar racing, olympic wrestling), in color.

On the 3 different sports, the task is very easy for the model. In color, it recognizes the 15 validation images from epoch 5, and the accuracy quickly reaches 100 %.

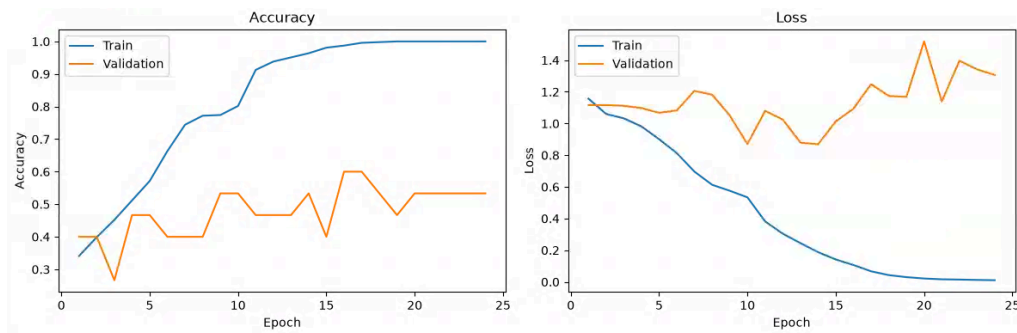


Fig. 6. – 3 similar sports (football, rugby, amputee football), in color.

On the 3 similar sports, it is the opposite. The model seems to learn the training set by heart, but the validation loss barely decreases and quickly goes back up: the validation accuracy stays around 53 % in color. The model does not manage to distinguish classes that look alike.

4.6.2. Grayscale or color

We then take the example of the 6 different sports (football, nascar racing, olympic wrestling, surfing, archery, arm wrestling), and this is what we want to compare.

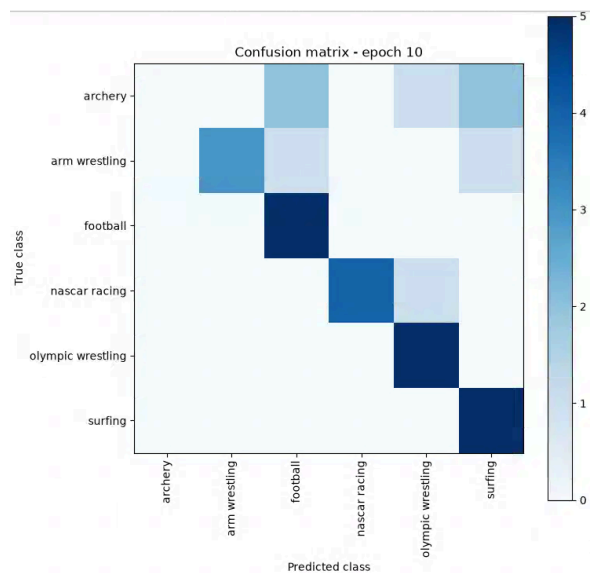


Fig. 7. – 6 sports in grayscale.

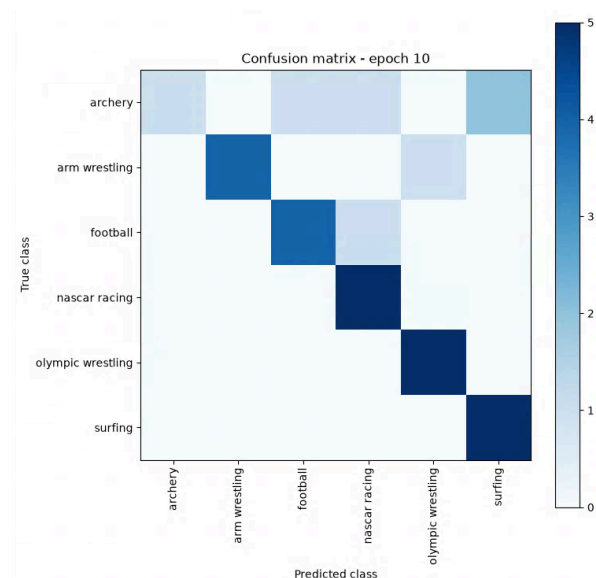


Fig. 8. – 6 sports in color.

On these 6 sports, color improves the validation loss (0.51 in grayscale against 0.40 in color), but the accuracy is almost the same. These sports are already quite easy to distinguish in grayscale.

The effect of color is much more visible when the task becomes harder, for example with 12 sports:

12 sports	Minimum validation loss	Accuracy at the best epoch
Grayscale	1.35	52 %
Color	0.87	78 %

Table F : Loss and accuracy got between grayscale and color dataset for 12 sports dataset

Our hypothesis is verified: color helps the model in all our subsets. This makes sense for sports, where the color of the field, the water or the outfits gives important clues.

4.6.3. A growing number of classes

With 20 sports, the validation accuracy drops to 49 % in grayscale and 58 % in color. With the 100 sports, it drops to 22 % in grayscale and 34 % in color. We were delighted to see that even with 100 sports, our model reached 34 % accuracy.

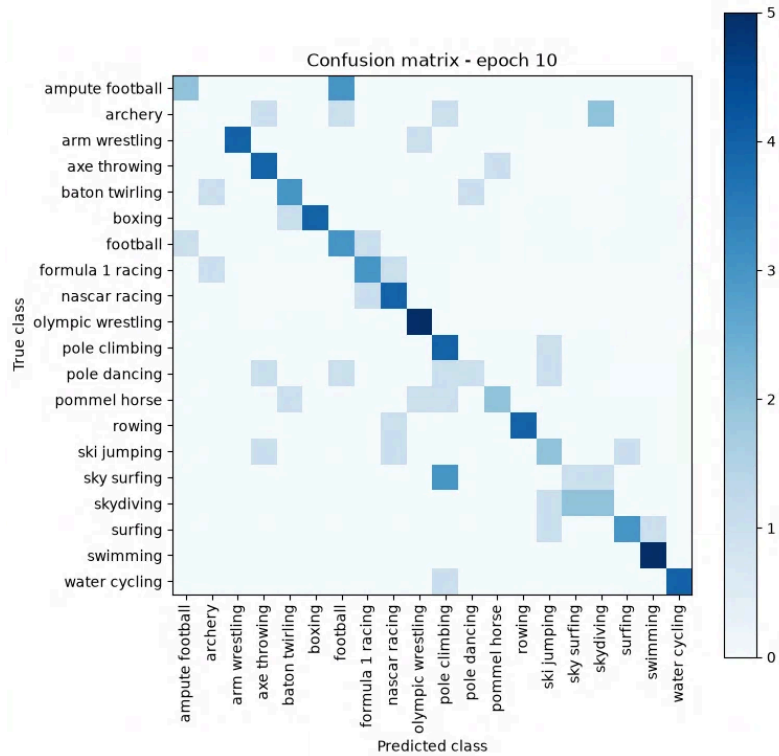


Fig. 9. – 20 sports in color: the diagonal is clearly visible, the model correctly recognizes part of the sports.

However, the overfitting is very strong: the best validation loss is reached as early as epoch 4, and then it rises sharply.

4.6.4. Summary

Sports	Random (1/n)	Grayscale	Color	Validation images (5 × n)
3 similar	33 %	40 %	53 %	15
3 different	33 %	93 %	100 %	15
6 different	17 %	80 %	83 %	30
12	8 %	52 %	78 %	60
20	5 %	49 %	58 %	100
100	1 %	22 %	34 %	500

Table G : Validation accuracy at the epoch with the lowest validation loss. The « Random » column gives the accuracy of a model that would answer at random.

4.7. Difficulties encountered

4.7.1. A first version that did not work

The process described above is actually our second version. In the first one, we had added regularization (a weight decay of 0.1) before switching from SGD to Adam, and then kept this same value.

The results were very bad: on 100 classes, the validation accuracy was about 1.6 %. The confusion matrices showed vertical lines: the model always predicted the same two or three classes, as if only a small part of the data was taken into account.

We changed the order of the steps as presented in this report: first the MLP and the convolution, then the optimizer and the scheduler, and only at the end the regularization. This solved most of the problems.

4.7.2. Other difficulties

- The validation sets are small, which makes the accuracy very unstable from one epoch to another.
- With 224×224 images, training takes a long time, which limits the number of possible experiments.
- **A code error:** for all the experiments on 12 sports, we had written `num_classes=20`. The model therefore had 8 outputs that did not correspond to any sport. We fixed the error, but its impact was limited.

5. Conclusions

- In terms of similar vs different sports: We saw that distinguishing very similar-looking classes is not a simple task. Our model seems unable to perform this task with confidence, as opposed to more different-looking classes, where the model performs with no issues.
- In terms of color: Switching to full color gave a substantial improvement. Grayscale discards chromatic information (hue and saturation), which removes cues that help distinguish classes and provides scene-level context, such as the green of a football pitch or the blue of water in water sports. The network can partially recover this from shape and texture, but not entirely, which is consistent with the gap we observed.
- In terms of class number: The challenge grows with the number of classes for two reasons. On one hand, just by increasing the number of classes, we increase the complexity of the problem (the chance of randomly guessing decreases). On the other hand, some of the classes we are adding are very similar to each other, and as we saw in our experiments, that also has an impact on the model's ability to predict with confidence.